

MAP COLORING (CSP)

AIM

To implement the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach in Python, assigning colors to regions such that no two adjacent regions have the same color

ALGORITHM

1. Start.
2. Define the map (graph) and available colors.
3. Select a region.
4. Assign a valid color to the region.
5. Check if any adjacent region has the same color.
6. If valid, move to the next region; otherwise, try another color.
7. Repeat until all regions are colored.
8. Display the color assignment.
9. Stop.

SAMPLE CODE

```
# Map Coloring using CSP (Backtracking) #
```

```
Graph representing neighboring states graph
```

```
= {
```

```
    'A': ['B', 'C', 'D'],
```

```
    'B': ['A', 'C'],
```

```
    'C': ['A', 'B', 'D'],
```

```
    'D': ['A', 'C']
```

```
}
```

```
# Available colors colors =
```

```
['Red', 'Green', 'Blue']
```

```
# Dictionary to store color assignment
```

```

color_assignment = {}

# Check if assigning a color is valid
def is_safe(node, color):
    for neighbor in graph[node]:
        if neighbor in color_assignment and color_assignment[neighbor] == color:
            return False
    return True

# Backtracking function
def map_coloring(nodes, index):
    if index == len(nodes):
        return True

    node = nodes[index]
    for color in colors:
        if is_safe(node, color):
            color_assignment[node] = color

    if map_coloring(nodes, index + 1):
        return True

# Backtrack
    del color_assignment[node]
    return False

# Main
nodes = list(graph.keys())

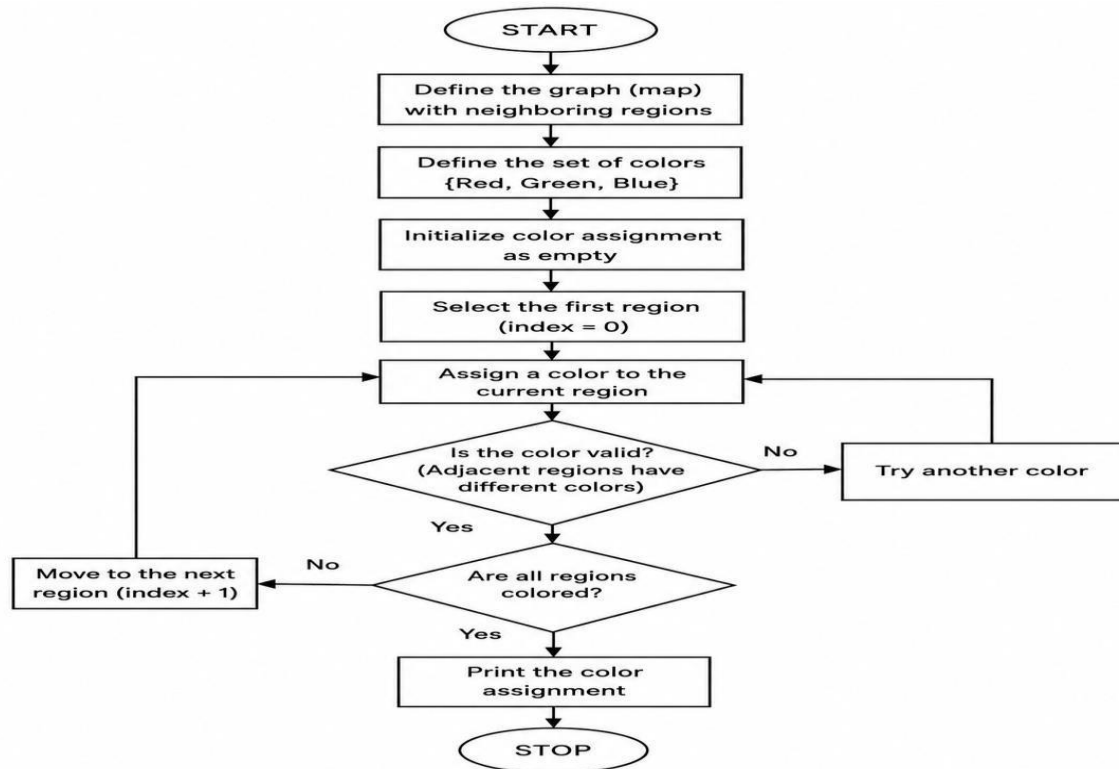
if map_coloring(nodes, 0):
    print("Color Assignment:")

    for node in nodes:
        print(f'{node} --> {color_assignment[node]}')
    else:

```

```
print("No solution exists.")
```

FLOW CHART



OUTPUT

```
Color Assignment:
A --> Red
B --> Green
C --> Blue
D --> Green

Process finished with exit code 0
```